

19.05.25

What's the general plan for this week?

1. Read and understand Jonas' code in detail.
2. Help him read and understand mine (there's still potential for improvement before he sits down to read mine).
3. Push mine to the condition where it can process large (40x40) test instances.
4. As a result of performance tests, decide which encoding we keep working on.
5. Implement speed (even in small environments, but at least to develop a strong concept).

—

IDEA: optimize waiting logic;

Currently it seems heavy and redundant, because I attempt to create waiting atoms for every track type. But this is clearly redundant. Why? Because when 'wait' is the action, there is no discrimination between the types - it doesn't really matter which type of track the train is standing on. Let's prove this:

1. We need to obtain "transition" atoms to see this:

```
transition(1,2,0,south,wait,0,south) transition(1,3,0,south,wait,0,south)
transition(1,4,0,south,wait,0,south) transition(1,5,0,south,wait,0,south)
```

Well, I was not quite right. The '*direction of waiting*' is important. The direction of waiting is defined by 'path' atoms. If we generate all possible directions of waiting for all possible types, it will ruin the beautiful compactness of my encoding. For example, on track type 1025 waiting can only be for the train heading east or west. However, let's just play around and see (I've created a copy of the 'motion' file for this, so it's safe).

Intuition: what if I create a 'path' for waiting from the 'rule-based' path recursively? Let's try to phrase it formally:

Normal rule idea	ASP phrasing	Code
Whenever there is a path atom holding a route description over a certain track type, there must be a path waiting atom over the same type where the entry position is simply doubled.	If there is a 'path' atom that implies movement over a certain track, then there must be a 'path' atom implying standing on this track.	<pre>path(IN_DIR, POSITION, wait, IN_DIR, POSITION):- path(IN_DIR, POSITION, _, _, _).</pre>

21.05.25

What small thing can I realistically implement now? It seems that this thing with creating 'path' atoms for waiting more elegantly has not been yet implemented. Let's look at the code and see if the draft of a normal rule from above has been implemented. Checking. Implemented - yes, but not tested: I think I haven't even run it once. Let's run it to see if this causes any errors / produces any output at all. Let's also switch to the simplest environment - straight line - for ease of debugging. The output is here. There is no need for waiting segments since there is no leeway. Let's give this leeway of 1. Yes, it works, just like with the less elegant declaration of waiting atoms that existed before. Let's now check the underlying "path" atoms - they are most likely correct, but just to make sure.

What are the possible next developments?

- a. A small improvement: making the position decoder more robust - generalizable to bigger environments (later, I need paper for it!).

Intuition: naively create the amount of waiting path atoms that corresponds to the indicated arrival time. That's naive, since it will increase grounding.

22.05.25

What should I do now? Well, now that I have a pen and paper, I can improve the positional converter.

Intuition: if need be, it's actually not so hard to rewrite the whole coordinate change to be in 2 dimensions (as in original), but it still seems reasonable to convert to 1 dimension to reduce grounding (every value is turned into a separate "constant" under the hood).

Let's at least test the grid size measurement first, including large environments (because we're aiming to generalize to large environments!).

Straight_line: 3x3 +

Г-turn: 3x3 +

П-turn: 3x3 +

Many turns: 4x4 +

Bowel for two: 4x4 +

Caduceus: 7x3 +

Large_test_1: 30x30 +

Large_test_2: 30x30 +

Large_test_3: 42x42 +

Good. The grid size gauge is correct. Now the next step. Let's again switch back to the smallest for development. Next thing, intuitively, is to count all cells. The counter is **correct**. Intuitively,

the next step is to create intermediate cells - cells that do what? I've counted, but this whole thing might be incorrect. I need to look at the numbers and try to understand the law they follow).

I have another intuition to check: the formula looks like

$x + y + \text{grid_size_dependent_constant}$

So, for a grid of 3x3 the constant is 2, for a grid of 4x4 the constant is 3, for a grid of 5x5 the constant is 4. The question is whether the constant arises from the grid's width or height. To find this out, I need to look at non-square grids. No, it's broken as we move up the rows.

Intuition:

$Y * \text{Num_columns} + X$

Testing. The formula seems correct, but somehow it doesn't work. Fixed (the bug was dumb - I used X twice instead of Y).

Testing on more instances:

All 3x3: **correct**.

Snake: **correct**. All others are correct too. **Success**.

—

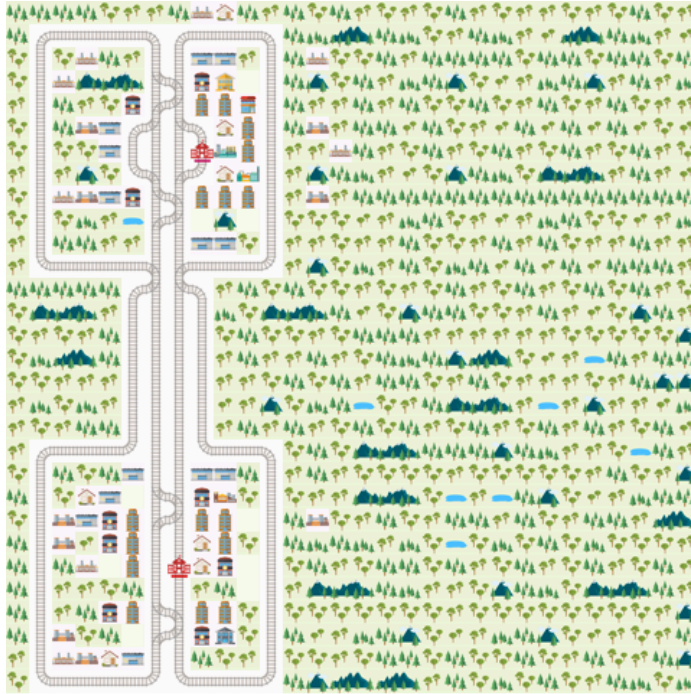
Let's now test the super large ones. Fails.

Debugging:

1. Standard inputs use one letter abbreviations for directions ('n' instead of 'north' etc).
Temporarily fixed in the input

H1: the way my waiting atoms are created (only one wait moment per position) prohibits formation of correct routes.

It would be useful to look at the image of the environment.



H2: some of the tracks used in the environment are not described in the 'path' atom creation.

Let's mark some of the suspects in the editor. But no! It's easier to check from inside the code by comparing the input with 'path' normal rules!

Yes: 37408 is not described!

Task: describe 37408. Doing: **Done** (it's still possible that it's bugged - will only be able to say after testing).

Debugging step: run trains one by one.

Result: None of the trains runs.

Still **fails**. Let's check if more types are undescribed: yes, 49186 is also undescribed!

Fixing: **fixed** (still may hold bugs). Testing the whole instance for 1 train again. **Fails**. Checking the types again: no, by now it seems that all types are in place.

H3: there is a mismatch in how Kep's environments vs my environments are described.

To check H3, let's look at bare path atoms. To get them, I'll also need to switch off constraints (one by one) that prohibit any answers now. Let's track this:

Constraint Disabled	Result (1 train)
Prohibiting clashes	fails
Addressing jumping over the grid problem	fails
The train must be directed correctly at the start	fails
The train must start from the departure position	works*
The train must arrive at the destination	...
Train must not jump between different cells, only moving by 1 cell at a moment of time	...

Interestingly, while for train 0 the encoding produced thousands of answers before I forced interruption, for train 1 there are only 2 answers. It's an unusual behaviour compared to what I've seen in previous projects! Let's play around with different combinations of constraints to see what's the *minimum number of disabled constraints that is required for the environment to run*. We'll backtrack through the same table.

Constraint Disabled	Result (1 train)
Prohibiting clashes	works Assumption: must not affect a single train in any way. Train 0: thousands Train 1: just 1 answer Intuition: due to the only constraint that's not in work being the one that sets the train into the starting position, the train just spawns at the destination and waits - hence 1 answer.
Addressing jumping over the grid problem	works Train 0: k of answers Train 1: just waits
The train must be directed correctly at the start	works The number of answers for train 1 reduced to 1 from 2. Still thousands of answers for train 0.
The train must start from the departure position	BLOCKS
The train must arrive at the destination	...
Train must not jump between different cells, only moving by 1 cell at a moment of time	...

OBSERVATION: the single answer that I receive for train 1 is actually bad, since it only waits:

```
action(1,0,wait) action(1,1,wait) action(1,2,wait) action(1,3,wait) action(1,4,wait) action(1,5,wait)
action(1,6,wait) action(1,7,wait) action(1,8,wait) action(1,9,wait) action(1,10,wait) action(1,11,wait)
```

```
action(1,12,wait) action(1,13,wait) action(1,14,wait) action(1,15,wait) action(1,16,wait) action(1,17,wait)
action(1,18,wait) action(1,19,wait) action(1,20,wait) action(1,21,wait) action(1,22,wait) action(1,23,wait)
action(1,24,wait) action(1,25,wait) action(1,26,wait) action(1,27,wait) action(1,28,wait) action(1,29,wait)
action(1,30,wait) action(1,31,wait) action(1,32,wait) action(1,33,wait)
```

For train 0, it's slightly better - some moves, but still a lot of waiting atoms. It's not clear why this happens or even *what it should look like*. Let's keep exploring.

What's the next debugging step? Steps?

In any order:

- Try to visualize the environment to be able to handle it better (the problem now is that there is no visual match between encoding and the image in terms of exact positions / types).
- Try running my own train(s) in the environment.
- Try gradually expanding from my current environment towards bigger sizes like 20x20 and 30x30.
- Asking Kep what can be principally different in his environment representations from how I implemented them (by giving examples) at the forum. This logically follows from c.

—

I'll go with c, since it seems to give me the most control over the process. Let's implement a 10x10 and then a 20x20 environment. It's also a good idea to 'mock' Kep's test environments to narrow the version space of *why it does it work*: if it doesn't work, then the problem is in some fundamental difference in representation - and it will be easy for Kep to answer at the forum.

Drafting a 10x10 environment.

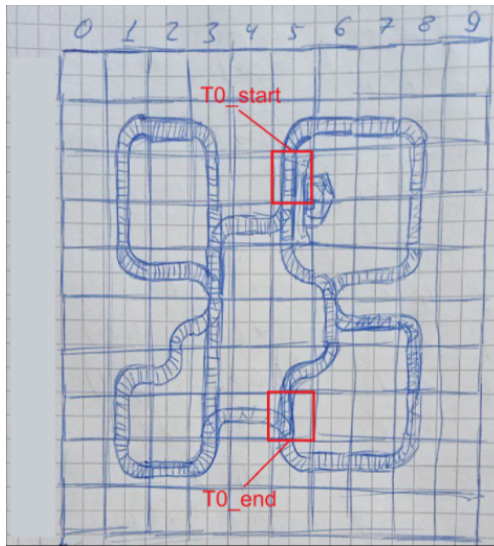


Describing it in 'cell' atoms.

Intuition: to do it faster, do it via short Python scripts. Developing one.
Done. Works. Now I only (only!) need to put in the types. **Done.**

Let's try running one train in this environment, increasing the difficulty step by step.

Test 1:



Let's explore the output: first, in "transition" atoms (they are easier to check):

```
transition(0,0,75,south,move_forward,65,south)
transition(0,1,65,south,move_forward,55,south)
transition(0,2,55,south,move_forward,56,east)
transition(0,3,56,east,move_forward,46,south)
transition(0,4,46,south,move_forward,36,south)
transition(0,5,36,south,move_forward,35,west)
transition(0,6,35,west,move_forward,25,south)
```

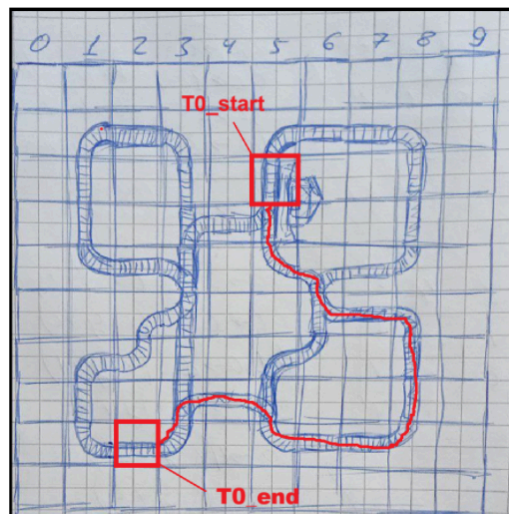
I don't have the 1d indexes for the whole route, but I can approximately track the movement. Let's at least check if the start and the end are exactly correct. The beginning is definitely correct. The last two segments are dodgy in terms of directions, although correct in

terms of indexes. Ah, no, I checked: they are perfectly correct. **Success.**

Let's now play around with starts and ends of the train, then implement two trains.

INTUITION: if I go to very big (30x30) environments of my own, making them work, while Kep's one is still failing, it means that there's a fundamental difference in how the environments are structured between my and Kep's encodings. It's now a big deal anyway, because my encoding is definitely robust (except for the 'wait' atoms).

Test 2:



We will only change the arrival cell:

```
end(0, (2, 5), 7). => end(0, (1, 2), 17).
```

No, 17 was an overkill! There is a shorter route that I didn't notice, and I'm getting *thousands* of answer sets (perhaps due to various permutations of 'wait' locations). The real shortest time seems to be 9:

```
end(0, (1, 2), 9).
```

Yes, then there is only one answer set:

Answer: 1

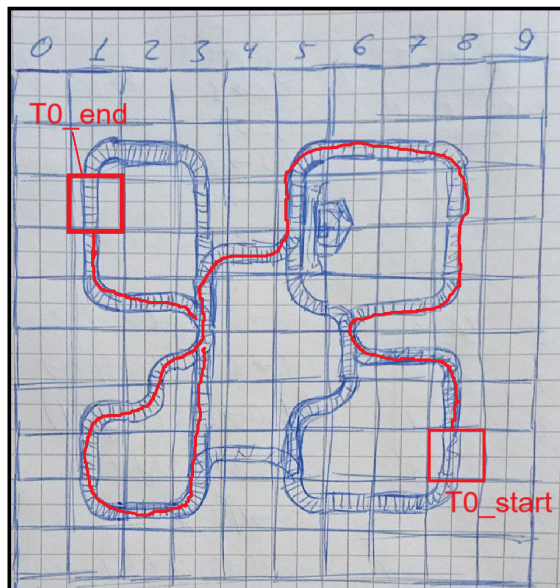
```
transition(0,0,75,south,move_forward,65,south) transition(0,1,65,south,turn_right,64,west)
transition(0,2,64,west,move_forward,63,west) transition(0,3,63,west,move_forward,53,south)
transition(0,4,53,south,move_forward,43,south) transition(0,5,43,south,move_forward,33,south)
transition(0,6,33,south,move_forward,23,south) transition(0,7,23,south,move_forward,13,south)
transition(0,8,13,south,move_forward,12,west)
```

In “action” atoms it also looks very good:

```
action(0,0,move_forward) action(0,1,turn_right) action(0,2,move_forward)
action(0,3,move_forward) action(0,4,move_forward) action(0,5,move_forward)
action(0,6,move_forward) action(0,7,move_forward) action(0,8,move_forward)
```

And it's definitely the shortest path. **Success.**

Test 3:



Here we change both the start and the end:

```
start(0, (2, 8), 0, north).
```

```
end(0, (7, 1), 34).
```

The time definitely may be bugged - be prepared for failure!

Answer: 1

```
transition(0,0,28,north,move_forward,38,north)
transition(0,1,38,north,move_forward,48,north)
transition(0,2,48,north,move_forward,47,west)
transition(0,3,47,west,move_forward,46,west)
transition(0,4,46,west,move_forward,56,north)
transition(0,5,56,north,turn_right,57,east)
transition(0,6,57,east,move_forward,58,east)
transition(0,7,58,east,move_forward,68,north)
transition(0,8,68,north,move_forward,78,north)
```

```
transition(0,9,78,north,move_forward,88,north) transition(0,10,88,north,move_forward,87,west)
transition(0,11,87,west,move_forward,86,west) transition(0,12,86,west,move_forward,85,west)
transition(0,13,85,west,move_forward,75,south) transition(0,14,75,south,move_forward,65,south)
transition(0,15,65,south,turn_right,64,west) transition(0,16,64,west,move_forward,63,west)
transition(0,17,63,west,move_forward,53,south) transition(0,18,53,south,move_forward,43,south)
transition(0,19,43,south,turn_right,42,west) transition(0,20,42,west,move_forward,32,south)
transition(0,21,32,south,move_forward,31,west) transition(0,22,31,west,move_forward,21,south)
transition(0,23,21,south,move_forward,11,south) transition(0,24,11,south,move_forward,12,east)
transition(0,25,12,east,move_forward,13,east) transition(0,26,13,east,move_forward,23,north)
transition(0,27,23,north,move_forward,33,north) transition(0,28,33,north,move_forward,43,north)
transition(0,29,43,north,move_forward,53,north) transition(0,30,53,north,turn_left,52,west)
transition(0,31,52,west,move_forward,51,west) transition(0,32,51,west,move_forward,61,north)
transition(0,33,61,north,move_forward,71,north)
```

Answer: 2

```
transition(0,0,28,north,move_forward,38,north) transition(0,1,38,north,move_forward,48,north)
transition(0,2,48,north,move_forward,47,west) transition(0,3,47,west,move_forward,46,west)
transition(0,4,46,west,move_forward,56,north) transition(0,5,56,north,turn_right,57,east)
transition(0,6,57,east,move_forward,58,east) transition(0,7,58,east,move_forward,68,north)
```

```

transition(0,8,68,north,move_forward,78,north) transition(0,9,78,north,move_forward,88,north)
transition(0,10,88,north,move_forward,87,west) transition(0,11,87,west,move_forward,86,west)
transition(0,12,86,west,move_forward,85,west) transition(0,13,85,west,move_forward,75,south)
transition(0,14,75,south,move_forward,65,south) transition(0,15,65,south,turn_right,64,west)
transition(0,16,64,west,move_forward,63,west) transition(0,17,63,west,move_forward,53,south)
transition(0,18,53,south,move_forward,43,south)
transition(0,19,43,south,move_forward,33,south) transition(0,20,33,south,move_forward,23,south)
transition(0,21,23,south,move_forward,13,south) transition(0,22,13,south,move_forward,12,west)
transition(0,23,12,west,move_forward,11,west) transition(0,24,11,west,move_forward,21,north)
transition(0,25,21,north,move_forward,31,north) transition(0,26,31,north,move_forward,32,east)
transition(0,27,32,east,move_forward,42,north) transition(0,28,42,north,move_forward,43,east)
transition(0,29,43,east,move_forward,53,north) transition(0,30,53,north,turn_left,52,west)
transition(0,31,52,west,move_forward,51,west) transition(0,32,51,west,move_forward,61,north)
transition(0,33,61,north,move_forward,71,north)

```

There are two answer sets that appear the same.

23.05.25

I'm currently moving toward environments that are similar to the ones suggested by Kep - in order to be able to conclude on the correctness of my encoding (in terms of the input above all).

The question now is: why did the last environment output two tracks? Let's compare the two outputs. An LLM can't reliably do so. Let's write a tiny Py script to compare two (or more) answers. **Done.**

I need to add to it the functionality of displaying the difference. The measured difference is:

0.7279040404040404

Which means that the two routes differ significantly. I've already spotted the difference. It's acceptable if it's defined by the possible tracks. Let's see. Now I can also display the difference:

```

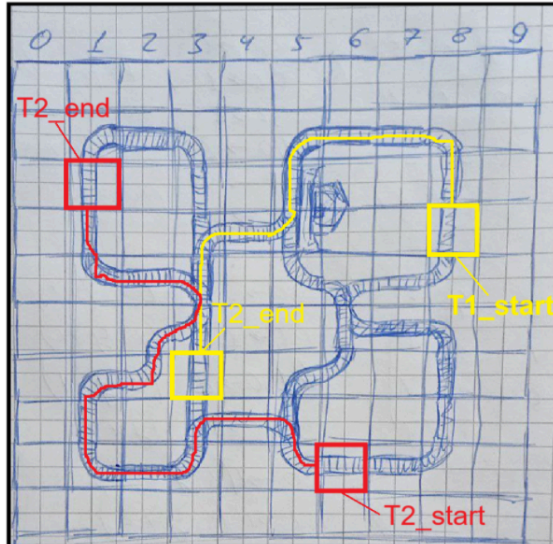
(0.7279040404040404, "replace: str1[905:1252] = 'turn_right,42,west)
transition(0,20,42,west,move_forward,32,south)
transition(0,21,32,south,move_forward,31,west)
transition(0,22,31,west,move_forward,21,south)
transition(0,23,21,south,move_forward,11,south)
transition(0,24,11,south,move_forward,12,east)
transition(0,25,12,east,move_forward,13,east)
transition(0,26,13,east,move_forward,23,north' vs str2[905:1257] =
'move_forward,33,south) transition(0,20,33,south,move_forward,23,south)
transition(0,21,23,south,move_forward,13,south)
transition(0,22,13,south,move_forward,12,west)
transition(0,23,12,west,move_forward,11,west)
transition(0,24,11,west,move_forward,21,north)
transition(0,25,21,north,move_forward,31,north)
transition(0,26,31,north,move_forward,32,east'")

```

Looks like a legally different transition. Let's follow quickly and move on. Let's assume the transition is legal and move on. One of the routes is definitely correct. **Success.**

Next step: run two trains in 10x10.

Test 4: two trains



The whole environment encoding stays the same. But I need to encode starts / ends for the two trains:

```
start(1, (6, 8), 0, north).  
end(1, (3, 3), 12).  
start(2, (1, 6), 0, west).  
end(2, (7, 1), 17).
```

First, let's run these two trains one by one to reveal potential basic mistakes.

Testing. **Empty set** for both trains together and separately. Why?! Let's debug by shrinking the route. Still the same. A suspicion: something is wrong with the solver. Try rebooting: it didn't help. Damn! I just *forgot to give trains themselves as facts!* **Fixed.**

The output for train 1:

```
transition(1,0,68,north,move_forward,78,north) transition(1,1,78,north,move_forward,88,north)  
transition(1,2,88,north,move_forward,87,west) transition(1,3,87,west,move_forward,86,west)  
transition(1,4,86,west,move_forward,85,west) transition(1,5,85,west,move_forward,75,south)  
transition(1,6,75,south,move_forward,65,south) transition(1,7,65,south,turn_right,64,west)  
transition(1,8,64,west,move_forward,63,west) transition(1,9,63,west,move_forward,53,south)  
transition(1,10,53,south,move_forward,43,south) transition(1,11,43,south,move_forward,33,south)
```

This route is perfectly correct. No waiting needed (or possible due to 0 leeway) for a single train.

The output for train 2:

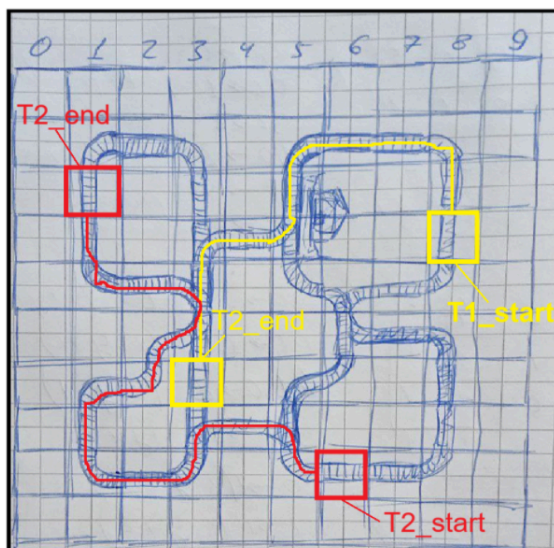
```
transition(2,0,16,west,move_forward,15,west) transition(2,1,15,west,move_forward,25,north)  
transition(2,2,25,north,turn_left,24,west) transition(2,3,24,west,move_forward,23,west)  
transition(2,4,23,west,move_forward,13,south) transition(2,5,13,south,move_forward,12,west)  
transition(2,6,12,west,move_forward,11,west) transition(2,7,11,west,move_forward,21,north)  
transition(2,8,21,north,move_forward,31,north) transition(2,9,31,north,move_forward,32,east)  
transition(2,10,32,east,move_forward,42,north) transition(2,11,42,north,move_forward,43,east)  
transition(2,12,43,east,move_forward,53,north) transition(2,13,53,north,turn_left,52,west)  
transition(2,14,52,west,move_forward,51,west) transition(2,15,51,west,move_forward,61,north)  
transition(2,16,61,north,move_forward,71,north)
```

Also perfectly correct. **Success.**

Now let's run them together (anticipation: no waiting needed, since they are not likely to meet at the only intersection they have). Testing: works:

```
transition(1,0,68,north,move_forward,78,north) transition(1,1,78,north,move_forward,88,north)
transition(1,2,88,north,move_forward,87,west) transition(1,3,87,west,move_forward,86,west)
transition(1,4,86,west,move_forward,85,west) transition(1,5,85,west,move_forward,75,south)
transition(1,6,75,south,move_forward,65,south) transition(1,7,65,south,turn_right,64,west)
transition(1,8,64,west,move_forward,63,west) transition(1,9,63,west,move_forward,53,south)
transition(1,10,53,south,move_forward,43,south) transition(1,11,43,south,move_forward,33,south)
transition(2,0,16,west,move_forward,15,west) transition(2,1,15,west,move_forward,25,north)
transition(2,2,25,north,turn_left,24,west) transition(2,3,24,west,move_forward,23,west)
transition(2,4,23,west,move_forward,13,south) transition(2,5,13,south,move_forward,12,west)
transition(2,6,12,west,move_forward,11,west) transition(2,7,11,west,move_forward,21,north)
transition(2,8,21,north,move_forward,31,north) transition(2,9,31,north,move_forward,32,east)
transition(2,10,32,east,move_forward,42,north) transition(2,11,42,north,move_forward,43,east)
transition(2,12,43,east,move_forward,53,north) transition(2,13,53,north,turn_left,52,west)
transition(2,14,52,west,move_forward,51,west) transition(2,15,51,west,move_forward,61,north)
transition(2,16,61,north,move_forward,71,north)
```

While hard to check, this is most likely just a combination (a 'superset') of the two previous answers for separate trains. No waiting was needed. Let's try to make them clash. How? By **making the train who's closer to the intersection lag by a corresponding amount of time units:**



```
start(1, (6, 8), 2, north).
end(1, (3, 3), 14).
start(2, (1, 6), 0, west).
end(2, (7, 1), 17).
```

Train 1: 10 moments till intersection

Train 2: 12 moments till intersection

Expected outcome: clash (unsatisfiable)

No, it gives me 4 different answers:

They imply different 'waiting' positions, so it all looks legit. Still, it would be useful to cause a

crash.

Let's switch to something easier for a while:

- Convert the journal to .md files and push to git.
- Get access to Jonas' encoding and try to understand ('internalize') it.

